Année Universitaire : 2025–2026
Institie : ISIMM
Matiére : Big Data
Sous la direction de Madamme : Asma Karkeni

Hydrolytics : Gestion Intelligente des Ressources en Eau Agricole

Pipeline Data Engineering End-to-End

Rapport Technique

HYDROLYTICS
SMART WATER MANAGEMENT

Auteur	Adib Ben HELAL
Module	Data Engineering — Master 1
Année	2025–2026
GitHub	https ://github.com/AdibBENHLEL/Hydrolytics
DockerHub	docker pull adibbh/hydrolytics

Stack technologique :

Apache Kafka PostgreSQL MinIO n8n Streamlit Docker Compose
Python Flask Pandas Open-Meteo API NASA POWER API

Table des matières

1 Contexte et Sources de Données **2**
1.1 Problématique . . . 2
1.2 Sources de Données . . . 2
1.3 Description des Parcelles Simulées . . . 2

2 Architecture Data et Justification du Stack **3**
2.1 Architecture Générale . . . 3
2.2 Schéma du Pipeline Annoté . . . 3
2.3 Justification du Stack Technologique . . . 4
2.4 Services Docker . . . 4

3 Transformations Principales Réalisées **5**
3.1 Transformation 1 — Calcul de l'Évapotranspiration (ETP) . . . 5
3.2 Transformation 2 — Indice de Stress Hydrique (0–10) . . . 5
3.3 Transformation 3 — Volume d'Irrigation Recommandé . . . 6
3.4 Résultats Observés . . . 6

4 Difficultés Rencontrées et Solutions Apportées **7**
4.1 Problème 1 — Connexion Kafka depuis Windows . . . 7
4.2 Problème 2 — MinIO Inaccessible depuis Windows . . . 7
4.3 Problème 3 — ETP Négative dans les Transformations . . . 7
4.4 Problème 4 — Score de Stress Hydrique Toujours à Zéro . . . 8
4.5 Problème 5 — IoT Simulator Crashait au Démarrage . . . 8
4.6 Problème 6 — Incompatibilité Versions Prefect . . . 8

5 Tests et Qualité des Données **9**
5.1 Stratégie de Tests . . . 9

6 Limites et Perspectives d'Amélioration **10**
6.1 Limites Actuelles . . . 10
6.2 Perspectives d'Amélioration . . . 10
6.2.1 Court terme (1–3 mois) . . . 10
6.2.2 Moyen terme (3–12 mois) . . . 10
6.2.3 Long terme (> 12 mois) . . . 10

1. Contexte et Sources de Données

1.1 Problématique

L'agriculture représente environ **70% de la consommation mondiale d'eau douce**. En Tunisie, les agriculteurs gaspillent jusqu'à 40% de l'eau d'irrigation par manque de données en temps réel sur l'état hydrique de leurs parcelles. Ce projet propose un pipeline Data Engineering complet qui centralise des données météorologiques, des mesures de capteurs IoT et des données satellitaires pour optimiser automatiquement l'irrigation agricole.

Objectif : Calculer en temps réel l'indice de stress hydrique de chaque parcelle agricole et déclencher automatiquement des alertes et recommandations d'irrigation personnalisées.

1.2 Sources de Données

Le pipeline exploite trois sources de données complémentaires, toutes gratuites et accessibles publiquement :

Source	Type	Données collectées	Fréquence
Open-Meteo API	Batch REST	Temp. max/min, précipitations, évapotranspiration FAO	Horaire
NASA POWER API	Batch REST	Rayonnement solaire, humidité atmosphérique	Horaire
IoT Simulateur	Streaming Kafka	Humidité sol, température, pH sol	Toutes les 5s

Table 1 – Sources de données du pipeline

Note : Les capteurs IoT sont simulés par un script Python publiant sur Kafka. Cette approche est une pratique standard en Data Engineering lors de la phase de développement, permettant de tester le pipeline sans matériel physique.

1.3 Description des Parcelles Simulées

Trois parcelles agricoles sont monitordées avec des profils d'humidité différents, représentatifs de situations réelles :

Parcelle	Humidité sol	pH sol	État typique
parcelle_1	25–45%	6.2	Stress modéré
parcelle_2	45–65%	6.8	Situation normale
parcelle_3	15–35%	7.1	Stress élevé

Table 2 – Profils des parcelles agricoles simulées

2. Architecture Data et Justification du Stack

2.1 Architecture Générale

Le pipeline suit une architecture **Data Lakehouse** en quatre couches fonctionnelles :

1. **Ingestion** — Collecte des données depuis les sources batch et streaming
2. **Stockage** — Data Lake brut (MinIO) et base structurée (PostgreSQL)
3. **Transformation** — Calculs métier et enrichissement des données
4. **Restitution** — Dashboard interactif en temps réel

2.2 Schéma du Pipeline Annoté

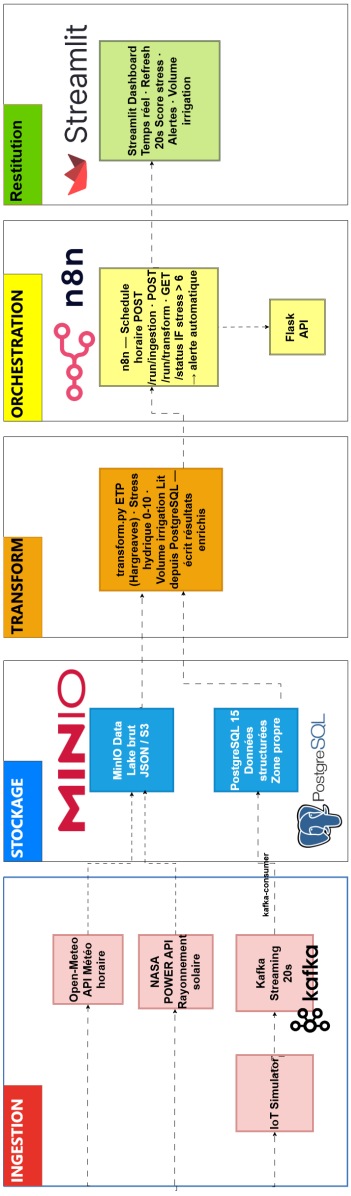

FIGURE 1 – Architecture du Pipeline

2.3 Justification du Stack Technologique

Couche	Technologie	Justification
Ingestion Batch	Python + requests	Léger, compatible toutes APIs REST, sans configuration
Streaming	Apache Kafka 3.7	Standard industrie pour le temps réel, haute disponibilité
Data Lake	MinIO	Compatible S3 AWS, dockerisable, 100% gratuit
Base de données	PostgreSQL 15	Relationnel, stable, excellent support Python
Transformation	Python + Pandas	Lisible, testable, transformations documentées
Orchestration	n8n	Interface visuelle drag & drop, scheduling intégré, retry natif
API Pipeline	Flask	Expose scripts Python pour n8n via HTTP
Visualisation	Streamlit	Python natif, déploiement rapide, refresh auto 5s
Déploiement	Docker Compose	Reproductible, portable, un seul fichier

Table 3 – Stack technologique et justifications

2.4 Services Docker

Le projet déploie **8 containers Docker** interconnectés via un réseau interne dédié (`irrigation-net`) :

Container	Image	Port	Rôle
kafka1	apache/kafka :3.7.0	9092	Bus de messages streaming
iot-simulator	custom Python	—	Simule 3 parcelles IoT
kafka-consumer	custom Python	—	Kafka → PostgreSQL
postgres	postgres :15	5432	Stockage données structurées
minio	minio/minio	9000/9001	Data Lake JSON bruts
pipeline-api	custom Flask	8000	API pour n8n
n8n	n8nio/n8n	5678	Orchestration automatique
streamlit	custom Python	8501	Dashboard interactif

Table 4 – Services Docker du projet

3. Transformations Principales Réalisées

Trois transformations métier identifiables ont été implémentées dans le fichier `transform.py`. Ces transformations combinent les données météo avec les données des capteurs IoT pour produire des indicateurs agronomiques exploitables.

3.1 Transformation 1 — Calcul de l'Évapotranspiration (ETP)

L'évapotranspiration représente la quantité d'eau perdue par le sol et les plantes combinées. Elle est calculée via la **formule de Hargreaves simplifiée** qui combine la température maximale, minimale et le rayonnement solaire NASA POWER.

Listing 1 – Calcul de l'ETP - Formule de Hargreaves

```python
def calcul_etp(temp_max, temp_min, rayonnement):
    """Formule de Hargreaves simplifiee
    Entrees : temperatures (C), rayonnement (MJ/m2/jour)
    Sortie  : ETP en mm/jour
    """
    if None in (temp_max, temp_min, rayonnement):
        return 0.0
    t_max = max(temp_max, temp_min)   # securite : ordre des
        temperatures
    t_min = min(temp_max, temp_min)
    diff  = max(t_max - t_min, 0.1)  # eviter racine de zero
    etp   = 0.0023 * diff**0.5 * (t_max + 17.8) * max(rayonnement, 1)
    return round(max(0, etp), 2)      # toujours positive
```

Formule de Hargreaves : $ETP = 0.0023 \times \sqrt{T_{max} - T_{min}} \times (T_{max} + 17.8) \times R_s$
Où R_s est le rayonnement solaire en MJ/m^2/jour.

3.2 Transformation 2 — Indice de Stress Hydrique (0–10)

Le score de stress hydrique est calculé à partir de l'humidité moyenne du sol mesurée par les capteurs IoT via Kafka. Plus l'humidité est basse, plus le score est élevé et plus l'alerte est critique.

Listing 2 – Calcul du score de stress hydrique

```python
def calcul_stress(etp, humidite_sol, pluie):
    """Score 0-10 base sur l'humidite du sol
    0-3  : situation normale
    4-6  : stress modere - surveiller
    7-10 : stress critique - irrigation immediate
    """
    if humidite_sol is None:
        humidite_sol = 50.0  # valeur par defaut
    if   humidite_sol < 20: score = 9.0   # critique
    elif humidite_sol < 30: score = 7.0   # eleve
    elif humidite_sol < 40: score = 5.0   # modere
    elif humidite_sol < 50: score = 3.0   # faible
    elif humidite_sol < 60: score = 1.5   # normal
    else:                   score = 0.5   # optimal
```

```
    return round(score, 2)
```

Humidité sol	Score	Niveau	Action recommandée
$< 20\%$	9.0	CRITIQUE	Irrigation immédiate requise
$20 - 30\%$	7.0	ÉLEVÉ	Irrigation urgente recommandée
$30 - 40\%$	5.0	MODÉRÉ	Surveillance accrue
$40 - 50\%$	3.0	FAIBLE	Aucune action immédiate
$50 - 60\%$	1.5	NORMAL	Situation satisfaisante
$> 60\%$	0.5	OPTIMAL	Pas d'irrigation nécessaire

TABLE 5 – Table de correspondance Humidité / Score de stress

3.3 Transformation 3 — Volume d'Irrigation Recommandé

Le volume d'irrigation est calculé pour ramener l'humidité du sol au niveau optimal de 60%. Il prend en compte la surface de la parcelle et le déficit hydrique actuel.

Listing 3 – Calcul du volume d'irrigation en litres

```
def calcul_volume(etp, pluie, humidite_sol, surface_m2=1000):
    """Volume d'eau necessaire pour atteindre 60% d'humidite
    Formule : deficit_humidite x surface x 10 litres/m2/%
    """
    if humidite_sol is None:
        humidite_sol = 50.0
    pluie = pluie or 0
    deficit_humidite = max(0, 60 - humidite_sol) / 100
    volume = deficit_humidite * surface_m2 * 10
    return round(volume, 2)
```

Exemple : Pour une parcelle à 23% d'humidité sur 1 000 m^2 :
$\text{Volume} = \dfrac{60 - 23}{100} \times 1000 \times 10 = \mathbf{3\,700}$ litres

3.4 Résultats Observés

Parcelle	Humidité moy.	ETP	Score stress	Alerte
parcelle_3	$\sim 23\%$	0.29 mm/j	7.0 / 10	**OUI**
parcelle_1	$\sim 33\%$	0.45 mm/j	5.0 / 10	Non
parcelle_2	$\sim 55\%$	0.44 mm/j	1.5 / 10	Non

TABLE 6 – Résultats des transformations pour les 3 parcelles

4. Difficultés Rencontrées et Solutions Apportées

4.1 Problème 1 — Connexion Kafka depuis Windows

Problème rencontré

Le consumer Kafka depuis Windows recevait les messages via `localhost:9092` mais Kafka retournait ensuite l'adresse interne `kafka1:9092` que Windows ne pouvait pas résoudre (*DNS failure — getaddrinfo failed*).

Modification de `KAFKA_ADVERTISED_LISTENERS` dans le `docker-compose.yml` pour utiliser `localhost:9092` au lieu de `kafka1:9092`, permettant aux clients Windows de se connecter directement sans résolution DNS interne Docker.

4.2 Problème 2 — MinIO Inaccessible depuis Windows

Problème rencontré

Les scripts Python lancés depuis Windows ne pouvaient pas atteindre MinIO car le hostname `minio` n'est résolvable que dans le réseau Docker interne (`irrigation-net`).

Utilisation de variables d'environnement avec `os.getenv()` permettant de basculer automatiquement entre `localhost` (exécution Windows) et `minio` (exécution Docker) selon le contexte d'exécution du script.

```
MINIO_CONFIG = {
    "endpoint_url": f"http://{os.getenv('MINIO_HOST', 'localhost')}:9000"
}
```

4.3 Problème 3 — ETP Négative dans les Transformations

Problème rencontré

L'API NASA POWER retournait parfois un rayonnement très faible et les températures min/max étaient inversées dans certains enregistrements, produisant des valeurs ETP négatives ou nulles incorrectes.

Ajout de gardes dans `calcul_etp()` : utilisation de `max()`/`min()` pour corriger l'ordre des températures, différence minimale de 0.1 pour éviter la racine carrée de zéro, et `max(0, etp)` pour garantir une valeur toujours positive.

4.4 Problème 4 — Score de Stress Hydrique Toujours à Zéro

Problème rencontré

La première formule de stress basée sur `ETP - pluie` donnait toujours 0 car l'ETP calculée était très faible (< 0.5 mm/jour en avril) et les précipitations étaient supérieures ou égales à l'ETP.

Reformulation complète du calcul de stress basée directement sur l'humidité du sol mesurée par les capteurs Kafka, indépendamment de l'ETP. Cette approche est plus directe, plus lisible et agronomiquement plus pertinente pour les conditions climatiques tunisiennes.

4.5 Problème 5 — IoT Simulator Crashait au Démarrage

Problème rencontré

Le container `iot-simulator` démarrait avant que Kafka soit prêt, causant un `KafkaTimeoutError` après 60 secondes et l'arrêt complet du container avec code d'erreur 1.

Ajout d'un `healthcheck` sur le service Kafka dans `docker-compose.yml` avec la condition `service_healthy`, et implémentation d'une boucle de retry dans `simulator.py` (20 tentatives, 5 secondes entre chaque).

```python
for attempt in range(20):  # retry 20 fois
    try:
        producer = KafkaProducer(bootstrap_servers=[KAFKA_BROKER])
        producer.bootstrap_connected()
        break
    except Exception as e:
        time.sleep(5)
```

4.6 Problème 6 — Incompatibilité Versions Prefect

Problème rencontré

Le client Prefect v3 installé sur Windows était incompatible avec le serveur Prefect v2 dans le container Docker, causant une `RuntimeError` : *client : 3.6.25, server : 2.20.25. Major versions must match.*

Décision stratégique de remplacer Prefect par **n8n** comme outil d'orchestration principal. n8n offre une interface visuelle drag & drop, un scheduling intégré, une gestion native des dépendances entre tâches et un système de retry sans nécessiter de code supplémentaire.

5. Tests et Qualité des Données

5.1 Stratégie de Tests

Dix-neuf tests unitaires ont été implémentés dans deux fichiers de test, couvrant les fonctions de transformation et la qualité des données produites.

Test	Description	Résultat
test_etp_valeurs_normales	ETP positive avec valeurs normales	PASSÉ
test_etp_valeur_none	ETP = 0 si valeur manquante	PASSÉ
test_etp_temperatures_inversees	ETP identique peu importe l'ordre	PASSÉ
test_stress_sol_tres_sec	Humidité $< 20\%$ = stress 9.0	PASSÉ
test_stress_sol_correct	Humidité $> 60\%$ = stress 0.5	PASSÉ
test_volume_sol_sec	Volume correct pour sol sec	PASSÉ
test_volume_sol_humide	Volume = 0 si sol humide	PASSÉ
test_alerte_declenchee	Alerte si stress > 6	PASSÉ
test_etp_range_realiste	ETP entre 0 et 20 mm/jour	PASSÉ
test_stress_range_valide	Score toujours entre 0 et 10	PASSÉ
test_coherence_stress_volume	Plus de stress = plus de volume	PASSÉ
	Total	**19/19**

Table 7 – Résultats des tests unitaires

Listing 4 – Commande d'exécution des tests

```
$ pytest tests/ -v
==================== 19 passed in 0.88s ====================
```

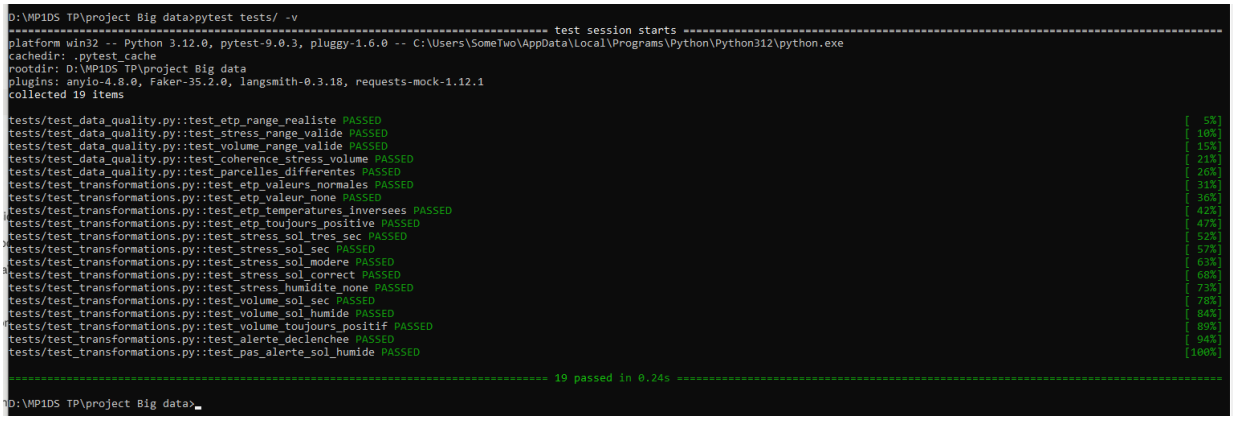

Figure 2 – Résultats des tests

6. Limites et Perspectives d'Amélioration

6.1 Limites Actuelles

Limite	Impact	Priorité
Capteurs IoT simulés	Données artificielles	Moyenne
Zone géographique unique (Tunis)	Portée géographique limitée	Faible
ETP faible en avril (saison)	Score stress sous-estimé	Faible
Pas de modèle prédictif ML	Pas d'anticipation du stress	Haute
Déploiement local uniquement	Pas d'accès distant	Haute
Pas de gestion multi-utilisateur	Usage individuel	Moyenne

Table 8 – Limites identifiées du projet

6.2 Perspectives d'Amélioration

6.2.1 Court terme (1–3 mois)

- **Hardware IoT réel :** Intégration de vrais capteurs Arduino/Raspberry Pi publiants directement sur Kafka
- **Données satellitaires :** Ajout d'images Sentinel-2 (ESA) pour la détection de stress végétal par NDVI
- **Alertes notifications :** Intégration d'alertes SMS/email via n8n en cas d'alerte critique
- **Ajout de parcelles :** Extension à 10+ parcelles avec localisation GPS réelle

6.2.2 Moyen terme (3–12 mois)

- **Machine Learning :** Modèle de prédiction du stress hydrique 48h à l'avance basé sur l'historique météo
- **Optimisation ETP :** Calibration des coefficients de Hargreaves avec des données locales tunisiennes
- **Déploiement cloud :** Migration vers AWS/GCP avec utilisation des crédits étudiants
- **API REST publique :** Exposition des recommandations via une API REST documentée (FastAPI + Swagger)

6.2.3 Long terme (> 12 mois)

- **Application mobile :** Interface mobile pour les agriculteurs (React Native)
- **Contrôle automatique :** Intégration avec des systèmes de vannes d'irrigation automates
- **Échelle régionale :** Déploiement sur l'ensemble de la région de Tunis (100+ parcelles)

— **Modèle économique :** Estimation des économies d'eau et ROI pour les agriculteurs

Ce projet démontre la maîtrise complète de la chaîne de valeur Data Engineering, depuis l'ingéstion de sources hétérogènes (batch + streaming) jusqu'à la restitution visuelle en temps réel. L'architecture Data Lakehouse choisie garantit la traçabilité des données brutes tout en offrant des données structurées pour l'analyse. Les sept difficultés techniques rencontrées ont toutes été résolues, renforçant la robustesse et la reproductibilité du pipeline. Les 19 tests unitaires valident la fiabilité des transformations métier. Ce système constitue une base solide pour une solution de gestion intelligente de l'eau agricole à échelle industrielle.